Feature-Bootstrapped Masked TSFM: Theory, Complexity, and Controlled Ablations for Reproducible Time-Series Pretraining

Vishn

***Abstract*—This paper presents a reproducible time-series pretraining pipeline and studies it at four levels: formal objective design, computational complexity, deterministic single-seed behavior, and multi-seed statistical behavior. The implementation trains a masked patch reconstruction model with RevIN, convolutional patch tokenization, a Transformer encoder backbone, and an MLP reconstruction head. The data loader supports two regimes: direct loading from raw `.tsf`/`.ts` files and feature-bootstrap synthesis from `*_features.csv` files. In the present repository state, training uses 44 feature tables and no raw TSF/TS files. We formalize the training objective, derive complexity with respect to context length and patch length, and report both deterministic and seed-stratified ablations over mask ratio and patch length. Multi-seed results indicate that patch length 32 is preferable to 16 and 8 under the tested budget, while an extreme single-seed patch-8 failure is not consistently reproduced across seeds. The paper emphasizes auditable claims, uncertainty reporting, and reproducibility instead of internet-scale zero-shot claims.**

***Index Terms*—time-series foundation model, masked reconstruction, transformer encoder, RevIN, synthetic pretraining, reproducibility**

I. Introduction

Large pretrained models changed natural language and vision workflows by turning pretraining into a reusable upstream stage [17], [16], [18], [19]. Time-series forecasting research now follows the same direction through patch-based transformers and foundation-model pretraining [1], [2], [4], [5], [6]. The central challenge is practical: many teams do not have internet-scale corpora, but they still need a principled pretraining recipe with transparent assumptions.

This paper studies a local TSFM-style pretraining pipeline implemented in a single training script. The implementation differs from decoder-only horizon prediction models such as TimesFM [1]: it trains an encoder with a masked latent reconstruction objective and a feature-bootstrap fallback path that synthesizes pseudo-series when raw files are absent.

The contribution is technical rather than promotional. We provide:

1) a formal problem statement and objective-level derivation for masked patch reconstruction in this codebase,
2) a complexity analysis that exposes patch-length trade-offs in attention and memory,
3) deterministic ablations over mask ratio and patch length with machine-readable artifacts.

We restrict claims to measured evidence from this repository. We do not claim state-of-the-art zero-shot forecasting.

II. Related Work

A. Foundation and Transfer Models for Forecasting

Early global forecasting models such as DeepAR [9], N-BEATS [10], and Temporal Fusion Transformer [11] established strong supervised baselines. Transformer variants then focused on long context and efficient sequence modeling, including Informer [4], Autoformer [5], FEDformer [6], DLinear [7], and iTransformer [8]. PatchTST showed that patch tokenization improves long-horizon transfer behavior [2]. TimesFM extended this direction to decoder-only foundation pretraining at larger scale [1].

B. Masked Modeling and Representation Learning

Masked prediction objectives were first standardized in language and then generalized to vision [16], [19]. The key principle is to force conditional reconstruction from partial context. Our objective follows the same family but reconstructs latent patch embeddings instead of tokens or pixels.

C. Normalization and Distribution Shift

Distribution shift in temporal amplitude and variance can degrade forecasting transfer. RevIN addresses this by instance-level normalization with reversible affine parameters [3]. This mechanism is central in our implementation because synthetic fallback data and heterogeneous datasets induce scale mismatch.

D. Synthetic Data for Time Series

Synthetic generation from stylized temporal components has long roots in classical time-series analysis [23]. Recent forecasting papers also mix real and synthetic corpora to improve coverage of pattern space [1]. Our work studies a constrained version of this idea using feature tables as synthesis control signals.

III. Methodological Positioning Relative to Decoder-Only TSFMs

Our implementation and TimesFM share a patch-based representation view, but they optimize different targets. TimesFM trains a decoder-only model for future patch prediction [1]. This repository trains an encoder for latent reconstruction under random patch masking. Table I summarizes the methodological difference.

TABLE I
METHODOLOGICAL POSITIONING OF THIS WORK VS. DECODER-ONLY TIMESFM

Dimension	Decoder-Only TimesFM	This Repository
Primary target	Next/future patch forecasting	Masked latent patch reconstruction
Backbone direction	Causal decoder stack	Bidirectional encoder stack
Scale assumptions	Internet-scale mixed corpus	Local corpus with feature bootstrap fallback
Core claim type	Zero-shot forecasting accuracy	Objective behavior and reproducibility
Evaluation emphasis	Benchmark forecasting metrics	Deterministic optimization ablations

IV. PROBLEM SETUP AND NOTATION

Let $x_{1:T_n}^{(n)}$ denote univariate series n in corpus $\mathcal{D}$. For a fixed context length L and stride S, the dataset builder extracts windows

$$w_k^{(n)} = x_{kS+1:kS+L}^{(n)}, \quad k \in \{0, \ldots, K_n - 1\}, \tag{1}$$

where $K_n = \left\lfloor \frac{T_n - L}{S} \right\rfloor + 1$.

Each window is mapped to tensor $\mathbf{W} \in \mathbb{R}^{L \times 1}$. The pretraining objective is not future horizon prediction; it is masked latent reconstruction over patch tokens.

A. Assumptions

We make the following assumptions for analysis and interpretation.

1) **A1 (Finite second moments):** each training window has finite mean and variance.
2) **A2 (Window-local regularity):** after RevIN, local statistics are stable enough for shared encoder weights.
3) **A3 (Feature control validity):** synthesis features (mean, variance, autocorrelation proxy, trend, season period, spike proxy) encode usable first-order structure.
4) **A4 (Missing-value handling):** non-finite observations removed during loading do not introduce directional bias larger than the modeled variance.
5) **A5 (Deterministic protocol for baseline ablations):** fixed seed and single-thread feature synthesis remove run-order nondeterminism.
6) **A6 (Seed-stratified protocol for uncertainty):** varying both data-synthesis seed and model seed approximates run-to-run uncertainty in this constrained budget.

B. Notation Table

Table II defines symbols used throughout the derivations.

V. FEATURE-BOOTSTRAPPED CORPUS CONSTRUCTION

A. Raw Mode

The loader searches recursively for `.tsf` and `.ts` files and uses `sktime` readers when available. Numeric extraction converts each object to finite `float32` arrays.

TABLE II
NOTATION

Symbol	Meaning
L	Context length in time points
S	Sliding-window stride
P	Patch length
$N = L/P$	Number of patch tokens
d	Embedding dimension
ρ	Mask ratio
$\mathbf{m}$	Binary mask over tokens
M	Number of Transformer encoder layers
B	Batch size
$\hat{\mathbf{E}}, \mathbf{E}$	Reconstructed and target patch embeddings

B. Feature-Fallback Mode

If no raw files exist, the loader reads `*_features.csv` rows and synthesizes pseudo-series. For each row, let target mean μ, target standard deviation σ, autocorrelation proxy a, linear trend slope b, seasonal period P, and spike controls (p_s, σ_s) define

$$\tilde{x}_t = \mu + bt + A \sin\left(\frac{2\pi t}{P}\right) + \epsilon_t + a r_t + \eta_t, \tag{2}$$

where ϵ_t is Gaussian noise, r_t is a random walk term, and η_t is sparse spike noise. The generated sequence is then rescaled to match target (μ, σ^2).

In this repository state, the corpus inventory is: raw TSF count = 0, raw TS count = 0, feature CSV count = 44.

VI. MODEL ARCHITECTURE

A. RevIN Layer

Given input window $\mathbf{W}$, RevIN computes

$$\mathbf{W}' = \gamma \odot \frac{\mathbf{W} - \mu(\mathbf{W})}{\sqrt{\sigma^2(\mathbf{W}) + \epsilon}} + \beta, \tag{3}$$

where (γ, β) are learnable affine parameters.

B. Patch Tokenization

For patch length P and context length L, token count is

$$N = \frac{L}{P}. \tag{4}$$

The implementation uses Conv1d with kernel and stride P to obtain patch embeddings $\mathbf{E} \in \mathbb{R}^{N \times d}$, then adds learned positional embeddings.

C. Masking and Encoder

Let mask vector $\mathbf{m} \in \{0, 1\}^N$ with $m_i \sim \text{Bernoulli}(\rho)$ for mask ratio ρ. Masked embeddings are

$$\mathbf{E}_i^{\text{mask}} = \begin{cases} \mathbf{e}_{\text{mask}}, & m_i = 1 \\ \mathbf{E}_i, & m_i = 0, \end{cases} \tag{5}$$

where $\mathbf{e}_{\text{mask}}$ is a learned mask token. A Transformer encoder stack maps $\mathbf{E}^{\text{mask}}$ to latent outputs, then an MLP reconstruction head predicts $\hat{\mathbf{E}}$.

VII. Objective, Optimization, and Stability

A. Masked Reconstruction Objective

The training loss is masked MSE on latent patch embeddings:

$$\mathcal{L}_{\text{mask}} = \frac{1}{\sum_i m_i} \sum_{i=1}^{N} m_i \left\| \hat{\mathbf{E}}_i - \mathbf{E}_i \right\|_2^2. \tag{6}$$

Define the full-token latent error $\ell_i = \|\hat{\mathbf{E}}_i - \mathbf{E}_i\|_2^2$. Under i.i.d. masking and concentration of $\sum_i m_i$, $\mathcal{L}_{\text{mask}}$ is a consistent estimator of the mean latent error over tokens. The estimator variance increases when ρ is very small and optimization difficulty increases when ρ is very large.

B. Estimator Property (Proof Sketch)

Let $\bar{\ell} = \frac{1}{N}\sum_{i=1}^{N} \ell_i$ and $\hat{\ell} = \frac{\sum_i m_i \ell_i}{\sum_i m_i}$. With independent Bernoulli masking, conditioned on $\sum_i m_i > 0$,

$$\mathbb{E}[\hat{\ell} \mid \ell_1, \ldots, \ell_N] \approx \bar{\ell}, \tag{7}$$

and the approximation tightens as N grows because denominator concentration improves.

The argument follows ratio-estimator concentration: numerator and denominator are sums of independent bounded random variables. In our setup, $N \in \{16, 32, 64\}$ for the tested patch lengths; therefore denominator variability is non-negligible but controlled for $\rho \in \{0.2, 0.4, 0.6\}$.

C. Gradient Accumulation

With accumulation factor K, micro-batch gradients are averaged before optimizer step:

$$\mathbf{g}_t = \frac{1}{K} \sum_{k=1}^{K} \nabla_\theta \mathcal{L}_t^{(k)}, \tag{8}$$

which reduces gradient variance by approximately $1/K$ under independent micro-batches.

Let $\mathbf{g}^{(k)}$ be unbiased per-micro-batch gradients with covariance Σ. Then

$$\operatorname{Var}\left[\frac{1}{K}\sum_{k=1}^{K} \mathbf{g}^{(k)}\right] = \frac{1}{K}\Sigma, \tag{9}$$

which explains the empirical stability gain from accumulation in low-memory regimes.

D. Stability Controls

The implementation uses AdamW, gradient norm clipping, optional AMP, and optional `torch.compile`. These controls target training stability and throughput rather than changing objective semantics.

VIII. Complexity Analysis

For batch size B, token count $N = L/P$, embedding width d, and encoder depth M:

$$\text{Cost}_{\text{layer}} = \mathcal{O}(BN^2 d + BNd^2), \tag{10}$$

$$\text{Cost}_{\text{encoder}} = \mathcal{O}\left(M(BN^2 d + BNd^2)\right). \tag{11}$$

Hence increasing patch length P decreases N and reduces the N^2 attention term. This gives a direct compute-speed advantage but can discard fine local dynamics when patches become too coarse.

Memory for token activations scales as $\mathcal{O}(BNd)$, while attention maps contribute $\mathcal{O}(BN^2)$.

A. Patch-Length Scaling Ratios

For fixed L and d, moving from patch length P_a to P_b changes token count from $N_a = L/P_a$ to $N_b = L/P_b$. The dominant attention ratio is

$$\frac{N_b^2}{N_a^2} = \left(\frac{P_a}{P_b}\right)^2. \tag{12}$$

Therefore, doubling patch length quarters the attention term. This ratio-level result predicts runtime trends but does not guarantee improved optimization because representational granularity also changes.

B. Compute-Normalized Objective View

For practical model selection under fixed hardware budgets, we define a compute-normalized criterion

$$\mathcal{J}(P, \rho) = \mathbb{E}[\mathcal{L}_{\text{mask}}(P, \rho)] + \lambda\, \mathcal{C}(P), \tag{13}$$

where $\lambda \geq 0$ is a budget preference scalar and

$$\mathcal{C}(P) \approx \alpha \left(\frac{L}{P}\right)^2 + \beta \left(\frac{L}{P}\right) \tag{14}$$

captures the dominant attention and projection terms. For two patch lengths P_a and P_b, P_b is preferred when

$$\Delta\mathbb{E}[\mathcal{L}_{\text{mask}}] < -\lambda\, \Delta\mathcal{C}, \tag{15}$$

with Δ denoting value at P_b minus value at P_a. This framing makes explicit that a slightly worse reconstruction loss can still be acceptable if compute savings are large enough for downstream throughput goals.

IX. Implementation-Level Algorithm

Algorithmically, one training step consists of the following stages.

1) Build a mini-batch of windows $\mathbf{W}$ from sliding-window dataset indices.
2) Apply RevIN normalization to each window instance.
3) Tokenize with Conv1d patch embedding and add learned positional embeddings.
4) Sample Bernoulli token mask and replace selected tokens with learned mask token.
5) Encode masked tokens with stacked Transformer encoder layers.

TABLE III
FIXED HYPERPARAMETERS FOR BOTH BASELINE AND MULTI-SEED ABLATIONS

Parameter	Value
Epochs	1
Max steps per epoch	20
Batch size	32
Context length L	512
Embedding dimension d	128
Heads / layers	8 / 4
Optimizer	AdamW (10^{-4}, wd 10^{-4})
Gradient accumulation K	4
Feature workers	1
Row cap per feature file	50
Checkpoint saving	Disabled

6) Reconstruct latent tokens with MLP head and compute masked MSE.
7) Accumulate gradients, clip global norm, and update with AdamW.
8) Persist per-epoch metrics to JSON artifacts when `--metrics-out` is set.

X. EXPERIMENTAL PROTOCOL

A. Single-Seed Deterministic Baseline

We first report a strict deterministic baseline (one seed, fixed settings) to establish direct comparability with earlier logs in this repository. The configuration is:

1) seed = 42,
2) feature fallback enabled,
3) `feature-workers=1` to avoid run-order nondeterminism,
4) `max-rows-per-feature-file=50`,
5) one epoch with `max-steps-per-epoch=20`,
6) no checkpoint saving during ablations.

These runs write structured JSON metrics to `experiments/ablations_det/`.

B. Multi-Seed Protocol for Uncertainty

The deterministic baseline is useful for reproducibility checks but insufficient for uncertainty estimation. We therefore run a seed-stratified protocol with seeds $\{11, 42, 123\}$ where each seed controls:

1) feature-row sampling in synthetic corpus construction,
2) model parameter initialization,
3) DataLoader shuffling order.

For each seed, the corpus is regenerated once and reused across the five ablation settings. This yields $n = 3$ replicates per setting with matched protocol and controlled budget.

Confidence intervals are reported as half-widths using Student-t:

$$\mathrm{CI}_{95}(\bar{x}) = t_{0.975,\, n-1} \frac{s}{\sqrt{n}}, \tag{16}$$

where $\bar{x}$ is the sample mean and s is the sample standard deviation.

TABLE IV
SINGLE-SEED BASELINE: MASK-RATIO ABLATION (PATCH LENGTH = 16)

Mask Ratio ρ	Steps	Masked MSE	Epoch Time (s)
0.2	20	0.3933	1.5883
0.4	20	0.4021	1.5940
0.6	20	0.4025	1.6166

TABLE V
SINGLE-SEED BASELINE: PATCH-LENGTH ABLATION (MASK RATIO = 0.4)

Patch P	Tokens N	Steps	Masked MSE	Epoch Time (s)	Token/s
8	64	20	3.72×10^{13}	2.8254	14497.09
16	32	20	0.4021	1.5940	12848.49
32	16	20	0.3835	1.0392	9853.74

C. Ablation Axes

We study two factors under fixed remaining hyperparameters:

1) mask ratio $\rho \in \{0.2, 0.4, 0.6\}$ at patch length $P = 16$,
2) patch length $P \in \{8, 16, 32\}$ at mask ratio $\rho = 0.4$.

XI. RESULTS

A. Single-Seed Baseline (Seed = 42)

Table IV and Table V reproduce the deterministic baseline exactly from `experiments/ablations_det/`. In that single-seed baseline, patch length 8 produced an extreme instability event.

B. Multi-Seed Mask-Ratio Analysis

Table VI shows that under $n = 3$ seed-stratified repeats, the three tested mask ratios are close in mean loss. The ordering slightly favors $\rho = 0.2$, but confidence intervals are wide relative to mean differences.

C. Multi-Seed Patch-Length Analysis

Table VII indicates that $P = 32$ gives the lowest mean masked MSE and the shortest epoch time among tested patch sizes. Patch length 8 remains the worst mean objective and slowest runtime.

D. Effect-Size View Relative to Baseline Configuration

Using $(P = 16, \rho = 0.4)$ as baseline, Table VIII reports relative effects from `experiments/multiseed_effects.csv`. Patch length changes dominate mask-ratio changes in this budget.

E. Outlier Reconciliation

The single-seed baseline recorded a catastrophic $P = 8$ outlier. The multi-seed protocol did not consistently reproduce that magnitude of failure. This suggests that the baseline divergence reflects a specific interaction among sampled synthetic rows, initialization, and short budget rather than an invariant property of $P = 8$. Practically, the more robust claim is weaker but still meaningful: $P = 8$ is less favorable than 16 and 32 under this budget, even when catastrophic instability is not consistently observed.

TABLE VI
MULTI-SEED MASK-RATIO RESULTS ($n = 3$, PATCH LENGTH = 16)

ρ	Mean MSE	Std	CI_{95} HW	Time (s)
0.2	0.3721	0.0356	0.2614	1.5550
0.4	0.3752	0.0375	0.2749	1.4751
0.6	0.3757	0.0385	0.2822	1.4642

TABLE VII
MULTI-SEED PATCH-LENGTH RESULTS ($n = 3$, MASK RATIO = 0.4)

P	N	Mean MSE	CI_{95} HW	Time (s)	Token/s
8	64	0.3934	0.4485	2.6162	15656.90
16	32	0.3752	0.2749	1.4751	13884.19
32	16	0.3651	0.1628	0.8941	11458.53

F. Compute-Normalized Interpretation

The empirical results are consistent with the objective in (13). Relative to baseline ($P = 16$), moving to $P = 32$ improves loss and reduces runtime. Moving to $P = 8$ increases runtime substantially and worsens mean loss. Therefore, under nonzero λ, the compute-normalized objective strongly prefers $P = 32$ in this experiment.

G. Stability-Efficiency Frontier

Mean loss alone is not sufficient for deployment-oriented selection. We therefore derive two additional diagnostics from the multi-seed aggregates:

$$\mathrm{CV}_c = \frac{s_c}{\bar{x}_c}, \tag{17}$$

$$\mathrm{SEI}_c = \frac{1}{\bar{x}_c\,\bar{t}_c}, \tag{18}$$

where $\bar{x}_c$ is mean masked MSE, s_c is its sample standard deviation, and $\bar{t}_c$ is mean epoch time for configuration c. Lower CV indicates greater seed-level stability; larger SEI indicates better joint objective-time efficiency.

Table IX shows three useful properties. First, patch-32 has the lowest variability (CV $= 0.0608$). Second, patch-32 yields the strongest joint efficiency (SEI $= 3.0638$), which is $1.696\times$ the baseline patch-16 setting and $3.153\times$ patch-8. Third, mask-ratio variants at patch-16 are nearly tied on both CV and SEI, reinforcing that patch length is the dominant axis under this budget.

XII. RESOURCE FOOTPRINT ANALYSIS

Beyond scalar epoch time, patch length changes the shape of the runtime bottleneck. Let H_a denote attention heads and f bytes per scalar (4 for `float32`). A first-order attention-map memory estimate per forward pass is

$$\mathcal{M}_{\text{attn}} \approx BMH_aN^2f. \tag{19}$$

This estimate ignores optimizer states and activation checkpointing, but it captures the patch-length trend because $N = L/P$ enters quadratically.

TABLE VIII
RELATIVE EFFECTS VS BASELINE ($P = 16, \rho = 0.4$)

Setting	ΔMSE (%)	Time Ratio
$\rho = 0.2,\ P = 16$	-0.84	1.05
$\rho = 0.6,\ P = 16$	$+0.12$	0.99
$\rho = 0.4,\ P = 8$	$+4.83$	1.77
$\rho = 0.4,\ P = 32$	-2.71	0.61

TABLE IX
DERIVED STABILITY AND EFFICIENCY METRICS FROM MULTI-SEED AGGREGATES

Setting	Mean MSE	CV	SEI
$P = 8, \rho = 0.4$	0.3934	0.1554	0.9717
$P = 16, \rho = 0.2$	0.3721	0.0958	1.7284
$P = 16, \rho = 0.4$	0.3752	0.0999	1.8067
$P = 16, \rho = 0.6$	0.3757	0.1024	1.8179
$P = 32, \rho = 0.4$	0.3651	0.0608	3.0638

For the fixed configuration ($B = 32, M = 4, H_a = 8, L = 512$), the attention-map contribution is shown in Table X. The ratio follows (12) exactly.

The throughput table and memory estimate together explain why patch-32 is favored in this budget: it reduces both attention memory pressure and epoch runtime while retaining enough representational capacity for the latent objective.

A. Patch-Dependent Parameterization and Fairness

Patch ablations are close to, but not exactly, iso-parameter in this implementation. The patch-dependent portion of model parameters is the sum of Conv1d patch embedding parameters and positional embedding parameters:

$$\Theta_{\text{patch}}(P) = d(P+1) + d\frac{L}{P}. \tag{20}$$

For $(L, d) = (512, 128)$, this gives $\Theta_{\text{patch}}(8) = 9344$ and $\Theta_{\text{patch}}(16) = \Theta_{\text{patch}}(32) = 6272$. The run artifacts confirm total-parameter counts of 835,586 for patch-8 and 832,514 for patch-16/32.

The parameter mismatch is very small, so the observed objective and runtime differences are primarily attributable to tokenization granularity and attention scaling, not to large capacity disparities.

XIII. STEP BUDGET VS. WALL-CLOCK BUDGET INTERPRETATION

All ablations in this study are step-matched: each setting runs exactly 20 optimizer steps. This isolates objective behavior per update but does not equalize wall-clock optimization opportunity. Because step time depends strongly on patch length, fixed-time conclusions can differ from fixed-step conclusions.

Let mean epoch time for configuration c be $\bar{t}_{\text{epoch},c}$ and epoch step count be S. Mean step time is

$$\bar{t}_{\text{step},c} = \frac{\bar{t}_{\text{epoch},c}}{S}. \tag{21}$$

TABLE X
APPROXIMATE ATTENTION-MAP MEMORY PER FORWARD PASS

Patch P	Token N	Relative N^2	$\mathcal{M}_{\text{attn}}$
8	64	16× vs $P = 32$	≈16 MB
16	32	4× vs $P = 32$	≈4 MB
32	16	1× baseline	≈1 MB

TABLE XI
MEASURED PARAMETER COUNTS ACROSS PATCH SETTINGS

Patch P	Tokens N	Total Params	Relative to $P = 16$
8	64	835,586	+0.369%
16	32	832,514	baseline
32	16	832,514	0.000%

For a wall-clock budget B seconds, expected available optimizer steps are

$$\mathcal{S}_{\text{wall}}(B, c) = \left\lfloor \frac{B}{\bar{t}_{\text{step},c}} \right\rfloor. \tag{22}$$

Using measured means from Table VII with $S = 20$, Table XII gives concrete budget translations.

At a 60-second budget, patch-32 receives about $1.65\times$ more updates than patch-16 and $2.93\times$ more than patch-8. Therefore, fixed-step and fixed-time evaluations answer different questions: fixed-step isolates per-update behavior, while fixed-time reflects practical training throughput. Reporting both perspectives is recommended for fair cross-configuration comparison.

XIV. DOWNSTREAM EVALUATION BLUEPRINT

The current study evaluates pretraining objective behavior. To connect these findings to forecasting utility, the next stage should evaluate downstream transfer with a fixed protocol:

1) **Checkpoint policy:** select checkpoints by lowest validation masked MSE and by fixed train-time budget.
2) **Transfer regimes:** compare linear probing, encoder-only fine-tuning, and full-model fine-tuning.
3) **Datasets:** include at least one hourly, daily, and weekly benchmark family with disjoint train/test entities.
4) **Horizons:** report short, medium, and long horizons (for example, 24/96/336 for hourly series).
5) **Metrics:** report MASE, sMAPE, and RMSE with horizon-wise breakdowns.

For consistency, we recommend computing

$$\text{sMAPE} = \frac{200}{H} \sum_{h=1}^{H} \frac{|y_{t+h} - \hat{y}_{t+h}|}{|y_{t+h}| + |\hat{y}_{t+h}| + \epsilon}, \tag{23}$$

and seasonal MASE

$$\text{MASE} = \frac{\frac{1}{H}\sum_{h=1}^{H} |y_{t+h} - \hat{y}_{t+h}|}{\frac{1}{T-m}\sum_{t=m+1}^{T} |y_t - y_{t-m}|}, \tag{24}$$

with season length m appropriate to sampling frequency.

This blueprint turns objective-level insights into benchmark-level evidence while preserving reproducibility discipline.

TABLE XII
EQUIVALENT OPTIMIZER STEPS UNDER FIXED WALL-CLOCK BUDGETS

Patch P	Steps in 30s	Steps in 60s	Steps in 120s
8	229	458	917
16	406	813	1627
32	671	1342	2684

TABLE XIII
FRIEDMAN RANK ANALYSIS FOR PATCH-LENGTH SETTINGS ($\rho = 0.4$)

Configuration	Rank Sum R_j	Mean Rank	Win Count
$P = 8$	7	2.33	1
$P = 16$	6	2.00	0
$P = 32$	5	1.67	2

XV. STATISTICAL AND METHODOLOGICAL VALIDITY

A. Internal Validity

We controlled optimizer, budget, architecture, and feature-processing settings. In the multi-seed study, each seed redefines both data sampling and initialization, so replicate variance includes both components.

B. Construct Validity

The dependent variable is masked latent MSE, not forecasting error. It captures representation reconstruction quality under masking and should not be conflated with direct horizon-forecast skill.

C. Precision Limits

With $n = 3$ seeds, confidence intervals are necessarily wide. The role of the current CI analysis is to prevent over-claiming and to identify which effects are large enough to survive small-sample uncertainty.

D. Matched-Seed Nonparametric Robustness Check

For patch-length settings at fixed $\rho = 0.4$, runs are matched by seed and can be analyzed with a Friedman test over per-seed ranks. Let N be the number of seeds, k the number of compared configurations, and R_j the rank sum for configuration j. The statistic is

$$Q = \frac{12}{Nk(k+1)} \sum_{j=1}^{k} R_j^2 - 3N(k+1). \tag{25}$$

Using $N = 3$ and $k = 3$ for $P \in \{8, 16, 32\}$ with lower masked MSE ranked better gives $(R_8, R_{16}, R_{32}) = (7, 6, 5)$, so $Q = 0.6667$. The corresponding concordance effect size is

$$W = \frac{Q}{N(k-1)} = 0.1111, \tag{26}$$

with a chi-square approximation $p \approx 0.7165$ (df $= 2$).

This test does not support strong inferential claims, which is expected at $N = 3$. Its value is methodological: it quantifies ranking consistency and confirms that the patch-32 advantage is directional but statistically underpowered in the current budget.

E. Seed-Count Planning

Given observed standard deviation s, target detectable mean difference δ, and approximate normal critical value $z_{0.975}$, a rough planning equation is

$$n \approx \left(\frac{z_{0.975}s}{\delta}\right)^2. \tag{27}$$

Using $s \approx 0.038$ from the mask-ratio repeats:

1) for $\delta = 0.01$, $n \approx 56$ seeds,
2) for $\delta = 0.005$, $n \approx 222$ seeds.

This back-of-the-envelope calculation explains why $n = 3$ can characterize large effects (for example patch-length shifts) but is underpowered for small mask-ratio deltas.

F. External Validity

The present results arise from feature-bootstrapped univariate pretraining in one repository state. External validity to multivariate settings, real-only corpora, and large-horizon zero-shot forecasting remains open.

XVI. Decision Analysis Under Small-Sample Uncertainty

The preceding sections report means, standard deviations, and configuration-wise confidence intervals. For model-selection decisions, pairwise differences versus baseline ($P = 16, \rho = 0.4$) are often more actionable. Let metric difference be

$$\Delta_{a,b} = \mu_a - \mu_b, \tag{28}$$

with approximate 95% interval

$$\mathrm{CI}_{95}(\Delta_{a,b}) = \Delta_{a,b} \pm t_{0.975,\nu}\sqrt{\frac{s_a^2}{n_a} + \frac{s_b^2}{n_b}}. \tag{29}$$

Using $n_a = n_b = 3$ and a conservative small-sample critical value, Table XIV summarizes uncertainty for both objective and runtime.

Two conclusions follow. First, objective-level pairwise differences remain uncertain at $n = 3$ because all masked-MSE intervals include zero. Second, runtime effects for patch changes are large and stable: patch-8 is reliably slower and patch-32 is reliably faster than baseline under the tested budget. In practical terms, patch-32 is currently justified by robust compute savings and favorable mean objective, while objective superiority alone should still be treated as provisional until seed count increases.

XVII. Discussion, Limitations, and Threats to Validity

A. Scope of Claims

The evidence supports objective-level and compute-level conclusions for this implementation. It does not establish benchmark-level superiority over decoder-only TSFMs á la TimesFM [1], [20].

B. Synthetic Data Bias

Feature-bootstrap synthesis preserves coarse descriptors but can miss local motifs, changepoints, and exogenous couplings. The resulting representation space may differ from one learned on raw observational streams.

C. Budget and Statistical Power

One epoch and 20 steps prioritize controlled comparisons over asymptotic convergence. The multi-seed extension improves robustness, but stronger inferential claims require larger seed counts and longer training horizons.

D. Design Implications

Within this budgeted setup, three implementation implications are immediate:

1) prioritize patch-length search before fine mask-ratio search,
2) treat catastrophic single-seed failures as hypotheses to test, not conclusions,
3) log machine-readable run metadata for every ablation to support post-hoc uncertainty analysis.

XVIII. Reproducibility Artifacts

The repository supports structured run logging through `--metrics-out`. Each JSON record stores configuration, system metadata, dataset statistics, epoch metrics, and summary metrics.

Representative baseline command:

```
py train_tsfm.py --data-dir data
--epochs 1 --max-steps-per-epoch
20 --feature-workers 1
--max-rows-per-feature-file 50
--mask-ratio 0.4 --patch-length 16
--no-save-checkpoint --metrics-out
experiments/ablations_det/mask_0p4.json
--run-name det_mask_0p4
```

Representative multi-seed command:

```
py run_multiseed_ablation.py
--data-dir data --output-dir
experiments/multiseed --summary-csv
experiments/multiseed_summary.csv
--seeds 11,42,123
--max-steps-per-epoch
20 --feature-workers 1
--max-rows-per-feature-file 50
```

Artifacts used in this paper:

1) `experiments/ablations_det/*.json`
2) `experiments/multiseed/*.json`
3) `experiments/multiseed_runs.csv`
4) `experiments/multiseed_summary.csv`
5) `experiments/multiseed_effects.csv`

TABLE XIV
APPROXIMATE PAIRWISE DIFFERENCE INTERVALS VS BASELINE ($P = 16, \rho = 0.4$)

Comparison	Masked MSE Difference (95% CI)	Epoch Time Difference in Seconds (95% CI)
$(P = 16, \rho = 0.2)$ vs baseline	-0.0032 $[-0.0860,\ 0.0797]$	$+0.0800$ $[-0.0305,\ 0.1905]$
$(P = 16, \rho = 0.6)$ vs baseline	$+0.0005$ $[-0.0856,\ 0.0865]$	-0.0109 $[-0.0381,\ 0.0164]$
$(P = 8, \rho = 0.4)$ vs baseline	$+0.0181$ $[-0.0968,\ 0.1330]$	$+1.1412$ $[+1.1018,\ +1.1805]$
$(P = 32, \rho = 0.4)$ vs baseline	-0.0102 $[-0.0800,\ 0.0596]$	-0.5810 $[-0.6185,\ -0.5435]$

TABLE XV
TOKEN BUDGET PER EPOCH FOR PATCH-LENGTH ABLATION

Patch Length P	Token Count N	$\mathcal{T}_{\text{tokens}}$
8	64	40960
16	32	20480
32	16	10240

XIX. CONCLUSION

This work presents a reproducible TSFM pretraining study that combines formal objective derivation, complexity analysis, deterministic baselines, and multi-seed uncertainty reporting. The main empirical result is that patch-length choice dominates mask-ratio choice in the tested budget, with $P = 32$ outperforming $P = 16$ and $P = 8$ on both objective quality and runtime. A severe single-seed $P = 8$ instability is observed but not consistently reproduced across seeds, reinforcing the need for seed-stratified reporting. The practical contribution is therefore methodological: an auditable pipeline for moving from objective-level diagnostics to downstream forecasting evaluation.

APPENDIX A
RATIO-ESTIMATOR VARIANCE ORDER

Let

$$\hat{\ell} = \frac{\sum_{i=1}^{N} m_i \ell_i}{\sum_{i=1}^{N} m_i}, \quad m_i \sim \text{Bernoulli}(\rho). \tag{30}$$

Using a first-order delta method on the ratio of sample means (A_N, B_N) with $A_N = \frac{1}{N}\sum m_i \ell_i$ and $B_N = \frac{1}{N}\sum m_i$, the leading variance term scales as

$$\text{Var}(\hat{\ell}) = \mathcal{O}\left(\frac{1}{N\rho}\right). \tag{31}$$

Thus, smaller token counts N (larger patches) and smaller mask ratios ρ both affect estimator variance in opposite directions through optimization and denominator concentration.

APPENDIX B
PROCESSED-TOKEN BUDGET ACCOUNTING

For one epoch with S optimizer steps, batch size B, and token count $N = L/P$, processed token count is

$$\mathcal{T}_{\text{tokens}} = SBN. \tag{32}$$

With $S = 20$, $B = 32$, and $L = 512$, this gives the accounting in Table XV.

APPENDIX C
RAW MULTI-SEED RUN MATRIX

Table XVI reports the per-run values used to build aggregate statistics. Publishing this matrix prevents hidden filtering and enables exact recomputation of means, standard deviations, and confidence intervals.

APPENDIX D
PATCH-8 STABILIZATION PLAYBOOK

The observed variability at $P = 8$ suggests a higher sensitivity region. A practical stabilization sequence is:

1) reduce base learning rate,
2) increase warmup steps,
3) increase gradient accumulation factor,
4) tighten gradient clipping threshold,
5) optionally reduce mask ratio while tuning.

One simple heuristic is to scale learning rate by token ratio:

$$\eta(P) = \eta(P_0)\sqrt{\frac{P}{P_0}}, \tag{33}$$

which lowers step size for smaller patches with larger token counts. This heuristic is not guaranteed optimal, but it is a practical starting point for reducing small-patch instability during early tuning.

APPENDIX E
METRICS SCHEMA AND PROVENANCE MAP

To make ablation claims independently auditable, every run writes a JSON payload with a stable schema. Table XVII lists the core fields used in this paper.

This schema enables three independent validations: (i) run-level reconstruction of tables, (ii) consistency checks between fixed-hyperparameter declarations and actual run configs, and (iii) seed-level provenance tracing.

APPENDIX F
SEED-LEVEL RANKING CONSISTENCY

Let $\mathcal{C}$ be the set of ablation configurations and $\mathcal{S}$ the seed set. For each seed $s \in \mathcal{S}$ and configuration $c \in \mathcal{C}$, define objective score $J_{c,s}$ as final masked MSE. The per-seed winner is

$$c_s^\star = \arg\min_{c \in \mathcal{C}} J_{c,s}. \tag{34}$$

Configuration-level win count is

$$w_c = \sum_{s \in \mathcal{S}} \mathbb{I}[c_s^\star = c]. \tag{35}$$

TABLE XVI
PER-RUN METRICS USED FOR AGGREGATE STATISTICS

Run	Seed	Data Seed	P	ρ	Masked MSE	Epoch Time (s)
ms_mask_0p2_s11	11	11	16	0.2	0.332700	1.6195
ms_mask_0p2_s42	42	42	16	0.2	0.402134	1.4827
ms_mask_0p2_s123	123	123	16	0.2	0.381352	1.5629
ms_mask_0p4_s11	11	11	16	0.4	0.333802	1.4808
ms_mask_0p4_s42	42	42	16	0.4	0.406754	1.4715
ms_mask_0p4_s123	123	123	16	0.4	0.385115	1.4729
ms_mask_0p6_s11	11	11	16	0.6	0.332419	1.4828
ms_mask_0p6_s42	42	42	16	0.6	0.406008	1.4529
ms_mask_0p6_s123	123	123	16	0.6	0.388628	1.4569
ms_patch_8_s11	11	11	8	0.4	0.323565	2.6357
ms_patch_8_s42	42	42	8	0.4	0.419061	2.5894
ms_patch_8_s123	123	123	8	0.4	0.437435	2.6237
ms_patch_32_s11	11	11	32	0.4	0.357278	0.8681
ms_patch_32_s42	42	42	32	0.4	0.390110	0.9026
ms_patch_32_s123	123	123	32	0.4	0.347816	0.9114

TABLE XVII
RUN-METRICS JSON SCHEMA USED IN THIS STUDY

Field Path	Semantics
`run_name`	Human-readable identifier; unique per ablation run.
`started_at`, `finished_at`	Wall-clock timestamps for run provenance and duration checks.
`config.seed`	Training seed controlling initialization and DataLoader order.
`config.data_seed`	Seed used for feature-row sampling in corpus synthesis.
`config.mask_ratio`	Bernoulli token masking probability ρ.
`config.patch_length`	Patch size P used by Conv1d tokenization.
`config.max_steps_per_epoch`	Fixed-budget ceiling for optimizer steps per epoch.
`system.device`	Execution backend (CPU/GPU) for reproducibility context.
`system.cpu_count`	Host CPU count used to contextualize throughput.
`data.source_mode`	Data mode indicator: raw or feature-fallback.
`data.loaded_files`	Number of files contributing usable series.
`data.loaded_series`	Count of resulting numeric series after cleaning/synthesis.
`data.total_windows`	Sliding-window sample count fed to DataLoader.
`model.total_params`	Total trainable parameter count for capacity tracking.
`epochs[*].steps`	Effective optimizer steps completed in epoch.
`epochs[*].masked_mse`	Objective value reported for masked latent reconstruction.
`epochs[*].epoch_time_sec`	Epoch runtime used for throughput and budget analysis.
`summary.total_train_time_sec`	End-to-end run time (not just epoch loop).
`summary.best_masked_mse`	Best objective value over recorded epochs.
`summary.final_masked_mse`	Final objective value used in ablation tables.
`summary.checkpoint_path`	Null in ablations (checkpoint saving disabled).

TABLE XVIII
SEED-LEVEL WINNING CONFIGURATION

Seed	Winning Configuration	Masked MSE
11	$P = 8, \rho = 0.4$	0.3236
42	$P = 32, \rho = 0.4$	0.3901
123	$P = 32, \rho = 0.4$	0.3478

Table XVIII reports the resulting seed-wise winners. While $P = 32, \rho = 0.4$ is not universally optimal, it wins in two out of three seeds and has the best aggregate mean.

This pattern is informative for practitioners: optimization landscapes at small budgets can be seed-sensitive, and reporting only a single deterministic run can overstate or understate a configuration's practical reliability.

REFERENCES

[1] A. Das, W. Kong, R. Sen, and Y. Zhou, "A decoder-only foundation model for time-series forecasting," arXiv:2310.10688, 2024.

[2] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *Proc. ICLR*, 2023.

[3] T. Kim, J. Baek, H. Song, Y. Choi, and others, "Reversible instance normalization for accurate time-series forecasting against distribution shift," in *Proc. ICLR*, 2022.

[4] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI*, 2021.

[5] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *Proc. NeurIPS*, 2021.

[6] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proc. ICML*, 2022.

[7] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?," in *Proc. AAAI*, 2023.

[8] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, and M. Long, "iTransformer: Inverted transformers are effective for time series forecasting," in *Proc. ICLR*, 2024.

[9] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," *Int. J. Forecasting*, vol. 36, no. 3, pp. 1181–1191, 2020.

[10] B. N. Oreshkin, D. Carpov, N. Chapados, and Y. Bengio, "N-BEATS: Neural basis expansion analysis for interpretable time series forecasting," in *Proc. ICLR*, 2020.

[11] B. Lim, S. Arik, N. Loeff, and T. Pfister, "Temporal fusion transformers for interpretable multi-horizon time series forecasting," *Int. J. Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021.

[12] R. Godahewa, C. Bergmeir, G. Webb, R. Hyndman, and others, "Monash

time series forecasting archive," in *Proc. NeurIPS Datasets and Benchmarks*, 2021.

[13] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M4 competition: Results, findings, conclusion and way forward," *Int. J. Forecasting*, vol. 34, no. 4, pp. 802–808, 2018.

[14] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "The M5 accuracy competition: Results, findings and conclusions," *Int. J. Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022.

[15] A. Vaswani *et al.*, "Attention is all you need," in *Proc. NeurIPS*, 2017.

[16] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL*, 2019.

[17] T. Brown *et al.*, "Language models are few-shot learners," in *Proc. NeurIPS*, 2020.

[18] A. Dosovitskiy *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021.

[19] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. CVPR*, 2022.

[20] N. Gruver, M. Finzi, S. Qiu, and A. Wilson, "Large language models are zero-shot time series forecasters," in *Proc. NeurIPS*, 2023.

[21] J. Kaplan *et al.*, "Scaling laws for neural language models," arXiv:2001.08361, 2020.

[22] J. Hoffmann *et al.*, "Training compute-optimal large language models," arXiv:2203.15556, 2022.

[23] G. Box, G. Jenkins, G. Reinsel, and G. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Hoboken, NJ, USA: Wiley, 2015.

[24] S. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.

[25] S. Bai, J. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," arXiv:1803.01271, 2018.

[26] Q. Wen, T. Zhou, C. Zhang, and L. Sun, "Time-series foundation models: A survey," arXiv:2310.10716, 2023.

[27] C. Bergmeir, R. Hyndman, and B. Koo, "A note on the validity of cross-validation for evaluating autoregressive time series prediction," *Computational Statistics & Data Analysis*, vol. 120, pp. 70–83, 2018.

[28] F. Petropoulos, N. Kourentzes, and G. Svetunkov, "Forecasting: Theory and practice in modern settings," *European Journal of Operational Research*, vol. 299, no. 1, pp. 1–14, 2022.